

Trabajo práctico Nro 2

Filsinger Gonzalo, Perea Ignacio, Leon Parfait Manuel

*Electrónica Aplicada II 4R2, Ingeniería Electrónica, Facultad Regional Córdoba
Universidad Tecnológica Nacional*

2. DESCRIPCIÓN DE LA MIGRACIÓN HACIA CMSIS

La migración consistió en reemplazar la instanciación de macros personalizadas con los punteros físicos a las direcciones de memoria (`#define RCC_APB2ENR *(volatile uint32_t *)0x40021018`) para utilizar estas direcciones ya definidas en la biblioteca del estándar del fabricante (`stm32f10x.h`).

- **Simplificación de escritura de registros:** Al llamar a `GPIOA->CRL` o a `ADC1->CR2`, el compilador calcula automáticamente el offset de memoria basándose en la dirección base del registro correspondiente.
- **Programación con palabras:** Se incorporó el uso de constantes predefinidas como `GPIO_BSRR_BS0` o `ADC_CR2_ADON`. Esto facilita a la hora de programar el calcular manualmente qué bit levantar (ej. `1 << 0`)) facilitando la comprensión visual del código.

la legibilidad mejora drásticamente, documentándose el código por sí mismo (`ADC1->CR2`).

- **Mantenimiento y Portabilidad:** Si el diseño exigiera migrar hacia otra línea de la familia STM32 (como F4 o G4), un desarrollo con punteros manuales requeriría reescribir prácticamente todo el código, ya que el mapa de memoria cambia. Con CMSIS, dado que la interfaz está estandarizada por ARM, funciones vinculadas al microcontrolador (como el control del NVIC o SysTick) son portables universalmente, y los periféricos mantienen su sintaxis estructural.
- **Organización:** Se obtienen las definiciones a los registros en una única cabecera del fabricante, previniendo errores de tipeo (como una dirección mal puesta) que al compilar no da error, pero no funcionaría la implementación física.

3. EXPLICACIÓN DE LA CONFIGURACIÓN DEL SYSTICK

- **Configuración:** Mediante la llamada a la función `SysTick_Config(SystemCoreClock / 1000);`, se cargó el registro de recarga (*Reload Value Register*) con la cantidad exacta de ciclos de reloj de CPU que caben en 1 milisegundo.
- **Operación en la Rutina de Interrupción (ISR):** Cada vez que la cuenta llega a cero, el hardware detiene el programa principal y ejecuta el handler `SysTick_Handler()`. Dentro de esta interrupción, se decrementan variables globales (como `msTicks`) e incrementan contadores continuos (como `globalTicks`).
- **Implementación de retardos:** Esto permitió crear una función de bloqueo determinista `Delay_ms()`, la cual, al atarse al decremento asíncrono modificado por hardware, garantiza una demora temporal estricta de hardware, posibilitando además la convivencia de tareas no bloqueantes (como el LED parpadeante continuo) junto con la lógica del conversor ADC.

4. ANÁLISIS COMPARATIVO: BAREMETAL PURO VS. CMSIS

- **Legibilidad:** El BareMetal puro te obliga a revisar el datasheet para buscar las direcciones de memoria correspondientes a sus respectivos registros. Utilizando CMSIS,

I. CODIGOS

I-A. main.c

```

1  \begin{lstlisting}[language=Linker]
2  #include "stm32f10x.h"
3
4  volatile uint32_t msTicks = 0;
5  volatile uint32_t globalTicks = 0;
6
7
8  volatile uint8_t flag_sensor_ext = 0;
9
10
11 void SysTick_Handler(void) {
12     if (msTicks > 0) {
13         msTicks--;
14     }
15     globalTicks++;
16 }
17
18 void EXTI0_IRQHandler(void) {
19     if(EXTI->PR & EXTI_PR_PR0) {
20         flag_sensor_ext = !flag_sensor_ext;
21         EXTI->PR = EXTI_PR_PR0;
22     }
23 }
24
25
26
27 void Delay_ms(uint32_t delay) {
28     msTicks = delay;
29     while(msTicks != 0); //
30
31 void Init_SysTick(void) {
32     SysTick_Config(SystemCoreClock / 1000);
33 }
34
35 void Init_GPIO(void) {
36     RCC->APB2ENR |= RCC_APB2ENR_AFIOEN |
37     RCC_APB2ENR_IOPAEN |
38     RCC_APB2ENR_IOPBEN |
39     RCC_APB2ENR_IOPCEN;
40
41     GPIOC->CRH &= ~(0xF << 20);
42     GPIOC->CRH |= (0x2 << 20);
43
44     GPIOA->CRL &= ~(0xFFFFF);
45     GPIOA->CRL |= 0x22222;
46
47     GPIOA->CRL &= ~(0xF << 20);
48
49
50     GPIOB->CRL &= ~(0xF << 0);
51     GPIOB->CRL |= (0x8 << 0);
52     GPIOB->ODR |= (1 << 0);
53 }
54
55 void Init_EXTI(void) {
56     AFIO->EXTICR[0] &= ~(0xF << 0);
57     AFIO->EXTICR[0] |= (0x1 << 0);
58
59     EXTI->IMR |= EXTI_IMR_MR0;
60     EXTI->FTSR |= EXTI_FTSR_TR0;
61
62
63     NVIC_EnableIRQ(EXTI0_IRQn);
64 }
65
66 void Init_ADC(void) {
67     RCC->APB2ENR |= RCC_APB2ENR_ADC1EN;
68
69     ADC1->CR2 |= ADC_CR2_ADON;
70     Delay_ms(1);
71
72     ADC1->CR2 |= ADC_CR2_TSVREFE;
73
74
75     ADC1->CR2 |= ADC_CR2_RSTCAL;
76     while(ADC1->CR2 & ADC_CR2_RSTCAL);
77     ADC1->CR2 |= ADC_CR2_CAL;
78     while(ADC1->CR2 & ADC_CR2_CAL);
79 }
80
81 uint16_t Read_ADC(uint8_t channel) {
82     ADC1->SQR3 = channel;
83
84
85     ADC1->CR2 |= ADC_CR2_SWSTART;
86
87     while(!(ADC1->SR & ADC_SR_EOC));
88
89     return ADC1->DR;
90 }
91
92 void Actualizar_LEDs_ADC(uint16_t adc_val) {
93     GPIOA->BRR = 0x1F;
94
95     if(adc_val > 682) GPIOA->BSRR = GPIO_BSRR_BS0;
96     if(adc_val > 1364) GPIOA->BSRR = GPIO_BSRR_BS1;
97     if(adc_val > 2046) GPIOA->BSRR = GPIO_BSRR_BS2;
98     if(adc_val > 2728) GPIOA->BSRR = GPIO_BSRR_BS3;
99     if(adc_val > 3410) GPIOA->BSRR = GPIO_BSRR_BS4;
100 }
101
102 int main(void) {
103     SystemInit();
104
105     Init_SysTick();
106     Init_GPIO();
107     Init_EXTI();
108     Init_ADC();
109
110     uint32_t last_led_toggle = 0;
111     uint16_t valor_adc = 0;
112
113     while (1) {
114         if (globalTicks - last_led_toggle >= 500) {
115             last_led_toggle = globalTicks;
116             GPIOC->ODR ^= GPIO_ODR_ODR13;
117         }
118
119         if (flag_sensor_ext) {
120             valor_adc = Read_ADC(5);
121         } else {
122             valor_adc = Read_ADC(16);
123         }
124
125         Actualizar_LEDs_ADC(valor_adc);
126
127         Delay_ms(50);
128     }
129 }

```

Listing 1. Configuración inicial en main.c